If the range can be divided, you first find the midpoint ❼ before using `std::async` to run the search in the second half of the range ❽, being careful to use `std::ref` to pass a reference to the `done` flag. In the meantime, you can search in the first half of the range by doing a direct recursive call ❾. Both the asynchronous call and the direct recursion may result in further subdivisions if the original range is big enough.

If the direct search returned `mid_point`, then it failed to find a match, so you need to get the result of the asynchronous search. If no result was found in that half, the result will be `last`, which is the correct return value to indicate that the value was not found ❿. If the "asynchronous" call was deferred rather than truly asynchronous, it will actually run here in the call to `get()`; in such circumstances the search of the top half of the range is skipped if the search in the bottom half was successful. If the asynchronous search is really running on another thread, the destructor of the `async_result` variable will wait for the thread to complete, so you don't have any leaking threads.

As before, the use of `std::async` provides you with exception-safety and exception-propagation features. If the direct recursion throws an exception, the future's destructor will ensure that the thread running the asynchronous call has terminated before the function returns, and if the asynchronous call throws, the exception is propagated through the `get()` call ❿. The use of a `try`/`catch` block around the whole thing is only there to set the `done` flag on an exception and ensure that all threads terminate quickly if an exception is thrown ⓫. The implementation would still be correct without it but would keep checking elements until every thread was finished.

A key feature that both implementations of this algorithm share with the other parallel algorithms you've seen is that there's no longer the guarantee that items are processed in the sequence that you get from `std::find`. This is essential if you're going to parallelize the algorithm. You can't process elements concurrently if the order matters. If the elements are independent, it doesn't matter for things like `parallel_for_each`, but it means that your `parallel_find` might return an element toward the